

Pergunta 1

Pontuou 2,400 de 6,000

Submission of the First Project - Extending a basic calculator

PT

- Paste ALL code from the Main module (i.e., your modified version of *Calculator.hs*). Do not submit or include any code from the *Parsing.hs* file.
- The code will be tested by calling the *main* function and then testing with some input from the standard input (stdin).
- You can download the PDF problem description, the implementation of the basic calculator and parsing module, and some sample input and output files from the Moodle home page.
- Make sure to check that your submission compiles. Programs that fail to compile will get zero marks.
- Make sure to run your program with the sample inputs and check the outputs.
- You will get partial marks for completing part 1 (sample 1) and part 2 (samples 2 and 3) of the problem description.
- There are extra private tests besides the 3 samples given.

```
calculator [] = return ()
calculator (l:ls) = do putStrLn (
                        calculator l
                        )

-- | evaluate a single expression
evaluate :: String -> String
evaluate txt
  = case parse expr txt of
      [ (tree, "") ] -> show (ev
      _ -> "parse error; try again"
```

Comentário:

Automatic ['O', 'O', 'X', 'X', 'O', 'O', 'X', 'X', 'X', 'X', 'O', 'O', 'O', 'O', 'X', 'X', 'X'] Matched pattern ['O', 'O', 'NOE', 'NOE', 'O', 'O', 'NOE', 'NOE', 'NOE', 'NOE', 'O', 'O', 'O', 'O', 'O', 'O', 'NOE', 'NOE', 'NOE'], assigned 0.4 of max grade. Comment: Part 1 only.

Informação

Do not import any modules in any of the questions.

If you are allowed to use functions from certain modules in certain questions, assume that the import statements are already written. DO NOT write them in the answers to the questions.

PT

Let us represent a set of connection between cities with distances using the types below:

```
type Node = String -- some city
type Dist = Int    -- some distance
type Edges = [(Node,Node,Dist)] -- directed connections
```

For example, the following edges represent some connections in Portugal (the distances are in kilometers).

```
portugal :: Edges
portugal = [ ("Porto", "Aveiro", 76)
            , ("Aveiro", "Coimbra", 63)
            , ("Aveiro", "Leiria", 117)
            , ("Coimbra", "Leiria", 76)
            , ("Leiria", "Santarem", 83)
            , ("Santarem", "Lisboa", 82)
            ]
```

Pergunta 2

Pontuou 1,000 de 1,000

Define a function ***inverse* :: Edges → Edges** that reverses the edges, maintaining the distances. Keep the edge order of the input list.

PT

Example:

```
ghci> inverse [("A","B",10), ("C","D",15)]
[("B","A",10), ("D","C",15)]
```

Restriction: For this exercise, you must use list comprehensions. Do not define recursive functions and do not use higher-order functions.

```
inverse :: Edges -> Edges
inverse xs = [(b, a, c) | (a, b,
```

```
inverse :: Edges -> Edges
inverse edges = [(y,x,d) | (x,y,d)<-edges]
```

Comentário: Automatic ['O', 'O', 'O', 'O', 'O', 'O']

Pergunta 3

Pontuou 1,000 de 1,000

Define a function ***newEdges :: Edges → Edges***, that composes pairs of edges:

PT

* if there is an edge (a,b) with distance d and another (b,c) with distance d', then there will be new edge (a,c) with distance d+d';

* exception: don't add edges (a,a) from a node to itself.

The result list should contain only the new edges.

Examples:

```
ghci> newEdges [("A","B",10), ("B","C",15), ("C","D",20)]
[("A","C",25), ("B","D",35)]
ghci> newEdges [("A","B",10), ("B","A",15)]
[]
```

Restriction: For this exercise, you must use list comprehensions. Do not define recursive functions and do not use higher-order functions.

```
newEdges :: Edges -> Edges
newEdges xs = [(a, d, i+j) | (a,
```

```
newEdges :: Edges -> Edges
newEdges edges = [(x,z,d+d') | (x,y,d)<-edges, (y',z,d')<-edges, y'==y && x/=z]
```

Comentário:

Automatic ['O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O']

Pergunta 4

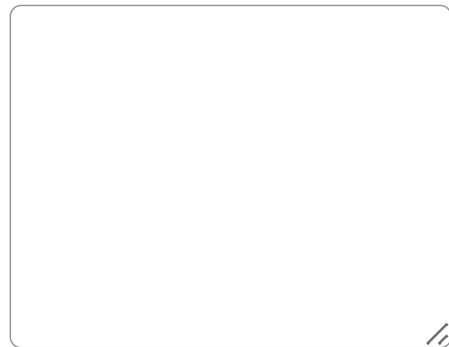
Pontuação 1,500

Define a function ***pathDistance*** :: *Edges* → [*Node*] → *Maybe Dist* that takes a list of edges and a list of nodes and computes the total distance of a path made up of the nodes in sequence (or Nothing if any of the connections in the path does not occur in the edges).

PT

Examples:

```
ghci> pathDistance [("A","B",10),("A","D",20),("B","C",15)] ["A","B","C"]
Just 25
ghci> pathDistance [("A","B",10),("A","D",20),("B","C",15)] ["A","C"]
Nothing
```



```
pathDistance :: Edges -> [Node] -> Maybe Dist
pathDistance edges [] = Just 0
pathDistance edges (start:path) = aux start 0 path
  where
    aux v acc [] = Just acc
    aux v acc (u:path) =
      case [d | (x,y,d)<-edges, x==v && y==u] of
        (d:_) -> aux u (acc+d) path
        [] -> Nothing
```

Pergunta 5

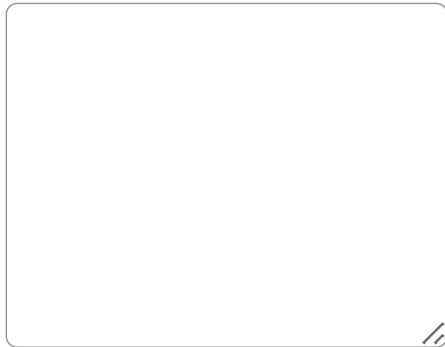
Pontuação 2,000

Define a recursive function ***shortest :: Edges → Edges*** that removes duplicated connections from a list of edges, keeping only the connection with minimum distance. Keep the edge order of the input list. In case of a tie, keep the first occurrence of the edge in the input list.

PT

Example:

```
ghci> shortest [("A","B",10), ("B","C",10), ("A","B",7), ("B","C",15)]
[("A","B",7), ("B","C",10)]
```



```
shortest :: Edges -> Edges
shortest [] = []
shortest ((x,y,d):edges) = (x,y,d'):shortest edges'
  where
    d' = minimum (d : [d' | (a,b,d')<-edges, a==x && b==y])
    edges' = [(a,b,d) | (a,b,d)<-edges, a/=x || b/=y]
```

Informação

Consider the ***Prop*** data type for propositions presented in the Lectures.

PT

```
type VarName = Char

data Prop = Const Bool
          | Variable VarName
          | Not Prop
          | And Prop Prop
          | Imply Prop Prop
          deriving Show
```

Pergunta 6

Pontuação 1,500

Define a recursive function ***rename*** :: ***[(VarName, VarName)]*** → ***Prop*** → ***Prop*** that renames all variables consistently using the list of pairs given as first argument.

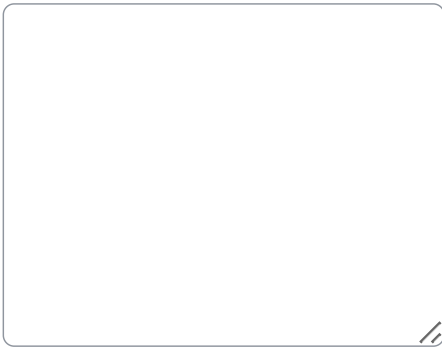
PT

If a variable does not occur in the list, then leave it unchanged.

Suggestion: use the lookup function from the Prelude, whose signature is ***lookup*** :: ***Eq a ⇒ a → [(a, b)] → Maybe b***.

Examples:

```
ghci> rename [('a','x'),('b','y')] (And (Variable 'a') (Variable 'b'))
And (Variable 'x') (Variable 'y')
ghci> rename [('a','b'),('b','a')] (And (Variable 'a') (Variable 'b'))
And (Variable 'b') (Variable 'a')
ghci> rename [('a','x')] (And (Variable 'a') (Variable 'b'))
And (Variable 'x') (Variable 'b')
```



```
rename :: [(VarName,VarName)] -> Prop -> Prop
rename _ (Const b) = Const b
rename ren (Variable x) = case lookup x ren of
    Just y -> Variable y
    Nothing -> Variable x
rename ren (Not p) = Not (rename ren p)
rename ren (And p q) = And (rename ren p) (rename ren q)
rename ren (Imply p q) = Imply (rename ren p) (rename ren q)
```

Pergunta 7

Pontuação 1,500

Define an auxiliary recursive function **nubBy** :: $(a \rightarrow a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$, that removes duplicate values from a list using the supplied argument function to check for equality.

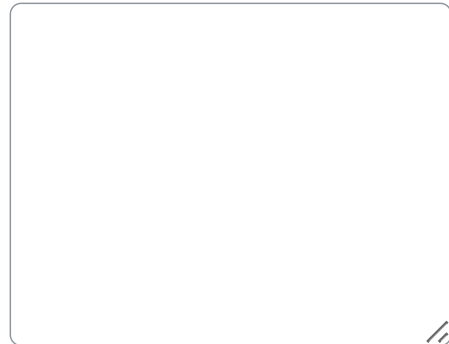
PT

In particular, the usual *nub* function is the same as *nubBy* (==).

For each value that appears multiple times in the input list, keep the first occurrence in the output list.

Examples:

```
ghci> nubBy (\(a,_) (c,_) -> a == c) [(3,3),(1,2),(3,1)]
[(3,3),(1,2)]
ghci> nubBy (==) "AbaaBCc"
"AbaBCc"
```



```
nubBy :: (a -> a -> Bool) -> [a] -> [a]
nubBy eq [] = []
nubBy eq (x:xs) = x : nubBy eq [x' | x' <- xs, not (eq x x')]
```

Pergunta 8

Pontuação 1,500

Using the *rename* function and *nubBy*, define a function ***normalize :: Prop → Prop*** that changes all variables in a proposition to '*a*', '*b*', '*c*', etc. consistently.

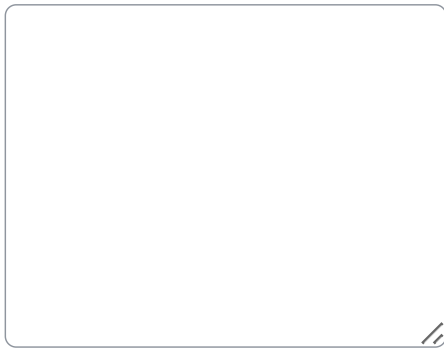
The variables' alphabetical order should match the order in which they first appear in the proposition if read as a String (from left to right).

Suggestion: define an auxiliary function *vars* to list all variables in a proposition (as was done in the lectures).

PT

Examples:

```
ghci> vars (And (Variable 'a') (Variable 'b'))
"ab"
ghci> vars (Imply (Variable 'x') (Variable 'x'))
"xx"
ghci> normalize (And (Variable 'p') (Variable 'r'))
And (Variable 'a') (Variable 'b')
ghci> normalize (Imply (Variable 'x') (Variable 'x'))
Imply (Variable 'a') (Variable 'a')
```



```
normalize :: Prop -> Prop
normalize p = rename ((flip zip ['a' ..] . findUniqueNames) p) p
  where
    findUniqueNames = nubBy (==) . vars

vars :: Prop -> [VarName]
vars (Const _) = []
vars (Variable x) = [x]
vars (Not p) = vars p
vars (And p q) = vars p ++ vars q
vars (Imply p q) = vars p ++ vars q
```

◀ Mini-Test 1 (2025/11/18) (Multiple-choice questions)

MT1 & TP1 Grades summary ▶